

SHL Conversational Assessment Recommender

1. OVERVIEW

A conversational REST API that accepts natural language hiring queries, retrieves relevant SHL assessments via a TF-IDF + FAISS pipeline, and returns structured JSON recommendations grounded in the catalog. Supports multi-turn conversations and asks clarifying questions for ambiguous inputs, and validates all recommendations against catalog entries to prevent hallucinations.

GitHub Repo: <https://github.com/ichindrasharma/shl-recommender>

2. SYSTEM ARCHITECTURE

Pipeline: User Query → FastAPI → TF-IDF + FAISS Retriever → Top-k Catalog Context → LLM (Llama 3.1 8B) → Structured JSON Response

Component	Role
FastAPI + Uvicorn	Async REST layer; routing and request validation
TF-IDF + FAISS	Vectorizes catalog at startup; cosine-similarity retrieval using L2-normalized TF-IDF vectors with FAISS IndexFlatIP (k=5)
SHL Catalog	Ground truth; enriched with role, skill, and seniority-level tags
Llama 3.1 8B	Generates structured JSON responses grounded in retrieved catalog context via strict prompting

3. RETRIEVAL & RAG PIPELINE

Dense transformer embeddings were intentionally avoided to reduce infrastructure overhead and maintain compatibility with free-tier deployment constraints. TF-IDF vectorization is applied to catalog entries at startup; queries are projected into the same vector space and FAISS performs cosine-similarity search. Top-k results (k=5) are injected as context into the LLM prompt. To compensate for TF-IDF's semantic limitations, catalog metadata was enriched with role keywords, job functions, seniority levels, and skill tags. Query expansion injects role context into the retrieval query before FAISS search.

4. PROMPT ENGINEERING

- **Structured output:** model constrained to JSON-only responses; schema enforced via system prompt
- **Catalog grounding:** recommendations must reference retrieved context; non-catalog names stripped post-generation
- **Clarification gate:** ambiguous queries trigger one targeted question before recommendations are issued
- **Output validation layer:** explicit instruction to avoid invented names; fuzzy containment check applied on output

Request/Response Schema

Request

```
{
  "message": "I need to hire a senior backend Python developer",
  "conversation_history": []
}
```

Response

```
{
  "reply": "Here are relevant assessments for a senior backend Python developer role.",
  "recommendations": [
    {
      "name": "Python (New)",
      "url": "https://www.shl.com/products/product-catalog/view/python-new/"
    },
    {
      "name": "Verify - Numerical Ability",
      "url": "https://www.shl.com/products/product-catalog/view/verify-numerical-ability/"
    }
  ],
  "end_of_conversation": false
}
```

5. API DESIGN

Method	Endpoint	Description
GET	/health	Health check. Returns {"status": "ok"}.
POST	/chat	Conversational recommendation endpoint
GET	/docs	Swagger/OpenAPI documentation

Conversation history is passed client-side, keeping the API fully stateless and horizontally scalable without server-side session storage.

6. EVALUATION

Role-based test scenarios: Backend Python Developer, AI/ML Engineer, Sales Executive, Customer Support Associate. Manual inspection and multi-turn stress testing.

- Precision of retrieved top-k assessments relative to query intent
- Manual relevance scoring of recommendations against role requirements
- Groundedness — every recommendation traced to a catalog URL
- Multi-turn consistency — system adapts correctly when constraints are added
- Hallucination rate — measured pre/post retrieval validation layer

Technical role retrieval (Python, ML) performed well. Sales and soft-skill roles required metadata enrichment to achieve comparable precision.

7. LIMITATIONS & MITIGATIONS

Issue	Mitigation
TF-IDF noisy on short/generic queries	Query expansion: role context injected before FAISS search
Recommendations clustered by type	Diversification enforced across test types in prompt
Over-filtering dropped valid results	Relaxed to fuzzy containment check
No seniority-level distinction in retrieval	Job-level tags added to catalog metadata

8. DEPLOYMENT

Platform	Render (free tier)
Stack	FastAPI + Uvicorn · Python
LLM	meta-llama/llama-3.1-8b-instruct via OpenRouter
Base URL	https://shl-recommender-0x7m.onrender.com
Endpoints	GET /health POST /chat GET /docs

Cold-start latency on Render's free tier is approximately 30–50 seconds after inactivity; subsequent requests respond promptly. The 8B model is sufficient for structured JSON generation within a retrieval-augmented context.

AI Assistance Disclosure: ChatGPT was used for debugging FAISS integration, Render deployment, and prompt engineering iteration. All architecture decisions, implementation, and evaluation were completed independently.